

**KULIAH PRAKTEK KOMPUTASI AWAN****MODUL 4.3****Dockerisasi Backend Kas RW***Containerization dan Distribusi via Docker Hub***Oleh: Dr. Andrew B. Osmond**

|                       |                                     |
|-----------------------|-------------------------------------|
| <b>Mata Kuliah</b>    | Praktikum Cloud Computing           |
| <b>Topik</b>          | Containerization dengan Docker      |
| <b>Studi Kasus</b>    | Aplikasi Pencatatan Kas RW          |
| <b>Perangkat</b>      | AWS EC2, Docker, Docker Hub, GitHub |
| <b>Estimasi Waktu</b> | 90 - 120 menit                      |

Pada modul ini, backend FastAPI yang sebelumnya dijalankan langsung di EC2 akan dibungkus ke dalam Docker container. Dengan containerization, seluruh environment aplikasi -- Python, library, dan konfigurasi -- dikemas menjadi satu image yang dapat dijalankan secara identik di mesin manapun hanya dengan perintah docker run.

**A. Tujuan Pembelajaran**

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- \* Menjelaskan konsep containerization dan perbedaannya dengan instalasi langsung
- \* Menulis Dockerfile untuk aplikasi Python FastAPI
- \* Melakukan docker build untuk membangun image dari Dockerfile
- \* Menjalankan container dengan docker run beserta konfigurasi environment
- \* Mendistribusikan image ke Docker Hub menggunakan docker push
- \* Menjalankan aplikasi di mesin baru hanya menggunakan docker pull dan docker run

**B. Latar Belakang**

Pada Modul 4.2, deployment backend memerlukan beberapa langkah setup: install Python, install pip, clone repo, install dependencies satu per satu. Proses ini rentan terhadap perbedaan versi, dependency conflict, dan harus diulang setiap kali ada server baru.

Docker hadir sebagai solusi dengan mengemas aplikasi beserta seluruh environment-nya ke dalam satu unit yang disebut container. Perbandingan pendekatan lama dan baru:

| <b>Aspek</b>                   | <b>Tanpa Docker (Modul 4.2)</b>                       | <b>Dengan Docker (Modul 4.3)</b> |
|--------------------------------|-------------------------------------------------------|----------------------------------|
| <b>Setup di server baru</b>    | Install Python, pip, clone repo, install dependencies | Install Docker, docker run       |
| <b>Konsistensi environment</b> | Bergantung pada versi Python di server                | Identik di semua mesin           |

|                            |                                                 |                               |
|----------------------------|-------------------------------------------------|-------------------------------|
| <b>Distribusi aplikasi</b> | Harus clone repo + setup ulang                  | docker pull dari Docker Hub   |
| <b>Dependency conflict</b> | Bisa terjadi jika ada paket sistem yang konflik | Terisolasi di dalam container |

### Analogi Docker Image

Docker image bukan sekadar kompresi folder aplikasi.

Image mengemas aplikasi BESERTA seluruh environment-nya -- Python, library, konfigurasi OS.

Lebih tepat dianalogikan sebagai file ISO instalasi yang sudah pre-configured:

tinggal 'boot' (docker run), langsung jalan tanpa setup apapun.

Ini yang dimaksud dengan 'membawa dependensinya sendiri'.

## C. Prasyarat

- \* Modul 4.2 sudah diselesaikan: backend berjalan di ec2-backend, frontend di ec2-frontend
- \* Repo kas-rw-backend sudah ada di GitHub
- \* Akun Docker Hub sudah dibuat di <https://hub.docker.com>
- \* EC2 instances masih berjalan dari sesi Modul 4.2

## D. Membuat Dockerfile

### D.1. Buat Dockerfile di Lokal

Di komputer lokal, masuk ke folder kas-rw-backend dan buat file Dockerfile:

#### macOS / Linux -- Terminal

```
cd ~/kas-rw-backend  
nano Dockerfile
```

#### Windows 11 -- PowerShell

```
cd $env:USERPROFILE\kas-rw-backend  
notepad Dockerfile
```

Isi Dockerfile dengan konfigurasi berikut:

```
FROM python:3.11-slim  
  
WORKDIR /app  
  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
  
COPY . .
```

```
EXPOSE 8000
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

### Penjelasan Setiap Instruksi Dockerfile

FROM python:3.11-slim : base image -- Python 3.11 versi minimal (slim).

WORKDIR /app : direktori kerja di DALAM container (dibuat otomatis, tidak perlu ada di lokal).

COPY requirements.txt : salin requirements.txt ke dalam container.

RUN pip install ... : install dependencies saat build (bukan saat run).

COPY . : salin seluruh kode aplikasi ke dalam container.

EXPOSE 8000 : dokumentasi bahwa container menggunakan port 8000.

CMD [...] : perintah yang dijalankan saat container di-start.

## D.2. Push Dockerfile ke GitHub

```
git add Dockerfile
git commit -m "feat: add Dockerfile for backend"
git push
```

## E. Build Docker Image di ec2-backend

### E.1. Install Docker

SSH ke ec2-backend, lalu install Docker:

```
sudo apt install -y docker.io
sudo systemctl start docker
sudo systemctl enable docker
```

### E.2. Pull Perubahan Terbaru

Ambil Dockerfile yang baru di-push dari GitHub:

```
cd ~/kas-rw-backend
git pull
ls
```

Pastikan Dockerfile muncul dalam daftar file.

### E.3. Build Image

```
sudo docker build -t kasrw-backend .
```

Proses ini memakan waktu beberapa menit karena perlu mengunduh base image Python. Setelah selesai, verifikasi image terbentuk:

```
sudo docker images
```

Output yang diharapkan:

| REPOSITORY    | TAG    | IMAGE ID     | CREATED       | SIZE   |
|---------------|--------|--------------|---------------|--------|
| kasrw-backend | latest | xxxxxxxxxxxx | X seconds ago | ~200MB |

### Image vs Container

Image : cetakan/template hasil docker build. Bersifat statis, tidak berubah.

Container : instance yang berjalan dari sebuah image, hasil docker run.

Satu image dapat menghasilkan banyak container yang berjalan secara bersamaan.

Flag -t pada docker build adalah 'tag' -- nama yang diberikan pada image.

## E.4. Jalankan Container

Stop uvicorn yang mungkin masih berjalan dari sesi sebelumnya (Ctrl+C), lalu jalankan container:

```
sudo docker run -d \  
  --name kasrw-backend \  
  -p 8000:8000 \  
  --env-file .env \  
  kasrw-backend
```

### Penjelasan Flag docker run

-d : detached mode -- container berjalan di background.

--name kasrw-backend : nama container (berbeda dari nama image).

-p 8000:8000 : port mapping -- forward port 8000 host ke port 8000 container.

--env-file .env : inject variabel environment dari file .env ke dalam container.

Verifikasi container berjalan:

```
sudo docker ps
```

Output yang diharapkan:

| CONTAINER ID           | IMAGE         | COMMAND                | STATUS       | PORTS |
|------------------------|---------------|------------------------|--------------|-------|
| xxxxxxxxxxxx           | kasrw-backend | "uvicorn main:app ..." | Up X seconds |       |
| 0.0.0.0:8000->8000/tcp |               |                        |              |       |

## E.5. Verifikasi

Buka browser dan akses Swagger UI:

```
http://<PUBLIC_IP_EC2_BACKEND>:8000/docs
```

Lakukan test POST /transaksi untuk memastikan container terhubung ke database. Buka juga frontend:

```
http://<PUBLIC_IP_EC2_FRONTEND>
```

Data transaksi harus tetap tampil -- backend kini berjalan di dalam Docker container.

## F. Distribusi via Docker Hub

### F.1. Login ke Docker Hub

Di terminal ec2-backend, login ke Docker Hub:

```
sudo docker login
```

Masukkan username dan password Docker Hub saat diminta.

### F.2. Tag Image

Tag image dengan username Docker Hub agar dapat di-push:

```
sudo docker tag kasrw-backend <username>/kasrw-backend
```

### F.3. Push ke Docker Hub

```
sudo docker push <username>/kasrw-backend
```

Setelah selesai, buka browser dan verifikasi image muncul di:

```
https://hub.docker.com/r/<username>/kasrw-backend
```

#### Nilai Pedagogis Docker Hub

Image yang sudah di-push ke Docker Hub dapat dijalankan siapapun di mesin manapun hanya dengan dua perintah -- tanpa perlu clone repo, install Python, atau install dependencies:

```
sudo docker pull <username>/kasrw-backend
```

```
sudo docker run -d --name kasrw-backend -p 8000:8000 --env-file .env <username>/kasrw-backend
```

Inilah portabilitas yang menjadi nilai utama containerization.

## G. Perintah Docker yang Berguna

| Perintah                                             | Fungsi                                           |
|------------------------------------------------------|--------------------------------------------------|
| <code>sudo docker images</code>                      | Melihat daftar image yang tersimpan              |
| <code>sudo docker ps</code>                          | Melihat container yang sedang berjalan           |
| <code>sudo docker ps -a</code>                       | Melihat semua container (termasuk yang berhenti) |
| <code>sudo docker stop kasrw-backend</code>          | Menghentikan container                           |
| <code>sudo docker start kasrw-backend</code>         | Menjalankan container yang berhenti              |
| <code>sudo docker rm kasrw-backend</code>            | Menghapus container                              |
| <code>sudo docker rmi kasrw-backend</code>           | Menghapus image                                  |
| <code>sudo docker logs kasrw-backend</code>          | Melihat log container                            |
| <code>sudo docker logs -f kasrw-backend</code>       | Melihat log container secara live                |
| <code>sudo docker exec -it kasrw-backend bash</code> | Masuk ke shell dalam container                   |

## H. Troubleshooting

### **docker: command not found**

Docker belum terinstal atau belum di-start. Jalankan:

```
sudo apt install -y docker.io
sudo systemctl start docker
```

### **Error: port is already allocated**

Port 8000 sudah dipakai oleh proses lain (kemungkinan uvicorn dari Modul 4.2 masih berjalan). Hentikan dulu:

```
# Cari proses yang memakai port 8000
sudo lsof -i :8000
# Hentikan container lama jika ada
sudo docker stop kasrw-backend
sudo docker rm kasrw-backend
```

### **Container berjalan tapi API tidak merespons**

Cek log container untuk melihat error:

```
sudo docker logs kasrw-backend
```

Kemungkinan penyebab: file .env tidak ditemukan atau DB\_HOST salah. Pastikan .env ada di direktori yang sama saat menjalankan docker run.

### **docker push ditolak (denied: requested access to the resource is denied)**

Pastikan sudah login (sudo docker login) dan nama image sudah di-tag dengan username Docker Hub yang benar sebelum push.

## **I. Kesimpulan**

Pada modul ini, backend Kas RW berhasil dibungkus ke dalam Docker container dan didistribusikan via Docker Hub. Siapapun kini dapat menjalankan backend Kas RW di mesin manapun hanya dengan Docker terinstal -- tanpa perlu menyentuh Python, pip, atau kode sumber sama sekali.

| Yang Sudah Dikerjakan                                                                                                                                                       | Yang Akan Datang                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Dockerfile dibuat dan di-push ke GitHub<br>Docker image dibangun di ec2-backend<br>Container berjalan menggantikan uvicorn langsung<br>Image didistribusikan via Docker Hub | Orkestrasi container dengan Kubernetes<br>Scaling dan load balancing<br>CI/CD pipeline otomatis |